

FULL STACK DEVELOPMENT

FASE 4 | AULA 01 -

FLAT LIST

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS	5
O QUE VOCÊ VIU NESTA AULA?	11
REFERÊNCIAS	12

EMANDA

O QUE VEM POR AÍ?

Nesta aula, exploraremos o componente FlatList do React Native, uma ferramenta essencial para renderizar listas de dados de forma eficiente e performática. Vamos entender sua importância, como configurá-lo corretamente e as melhores práticas para utilizá-lo em diferentes contextos.

No Hands On, implementaremos uma FlatList que renderiza uma lista de itens, com funcionalidades de atualização e carregamento dinâmico.

EXEMPLO

HANDS ON

Configuraremos uma FlatList em uma aplicação React Native para renderizar uma lista de dados de forma eficiente. Inicialmente, definiremos os dados que serão exibidos na lista e, em seguida, configuraremos a FlatList para renderizá-los.

Aprenderemos a personalizar a aparência dos itens da lista utilizando renderItem e a chave dos itens com keyExtractor, e implementaremos funcionalidades adicionais, como pull-to-refresh para atualizar os dados e infinite scrolling para carregar mais itens conforme o usuário rola a lista.

Discutiremos a importância de usar a FlatList em vez de mapear diretamente os componentes de lista para garantir melhor performance, especialmente em listas grandes. Por fim, abordaremos como otimizar a performance da FlatList utilizando propriedades como initialNumToRender e maxToRenderPerBatch, garantindo uma experiência de usuário suave e responsiva.

SAIBA MAIS

FlatList é um componente fornecido pelo React Native amplamente utilizado para renderizar listas de dados de maneira eficiente. Ele é ideal para exibir grandes quantidades de dados, pois utiliza técnicas de virtualização para renderizar apenas os itens visíveis na tela a qualquer momento. Isso não só melhora significativamente a performance da aplicação, como também reduz o consumo de memória, tornando o FlatList uma escolha perfeita para aplicações que lidam com listas longas ou complexas.

Por que usar FlatList?

Eficiência

A principal vantagem do FlatList é sua eficiência em lidar com grandes volumes de dados. Em vez de renderizar todos os itens de uma vez, ele renderiza apenas os itens que são visíveis na tela e alguns itens extras para suavizar a rolagem. Isso é alcançado por meio da virtualização, onde os itens fora da tela são desmontados e reconstruídos conforme necessário. Esta abordagem minimiza o uso de memória e CPU, resultando em uma performance significativamente superior em comparação com uma lista que renderiza todos os itens de uma vez.

Facilidade de uso

O FlatList fornece uma API simples e flexível que facilita a implementação de funcionalidades comuns em listas. A configuração básica do FlatList requer apenas dois parâmetros obrigatórios: data e renderItem. O data é a fonte dos itens a serem renderizados, e renderItem é uma função que define como cada item deve ser exibido. Além disso, o FlatList suporta funcionalidades avançadas, como rolagem infinita, cabeçalhos e rodapés de lista e atualização de dados em tempo real com relativa facilidade.

Personalização

Outro ponto forte do FlatList é a sua capacidade de personalização. Ele permite que os(as) desenvolvedores(as) personalizem completamente a aparência dos itens da lista. Você pode definir estilos específicos, adicionar headers, footers, separadores de

itens, e até mesmo controlar como os itens são renderizados por meio das propriedades `ItemSeparatorComponent`, `ListHeaderComponent` e `ListFooterComponent`. Isso proporciona uma flexibilidade enorme para adaptar a lista às necessidades específicas da sua aplicação.

Quando usar FlatList?

Ele é ideal para qualquer situação na qual você precise renderizar listas longas ou complexas de dados, como:

- Aplicações de feed de notícias e mídias sociais: aplicações que exibem posts ou artigos em um feed contínuo. Redes sociais e blogs se beneficiam enormemente da eficiência do FlatList, pois ao utilizar o FlatList, as redes sociais e blogs tem os comentários, postagens de usuários e atualizações de status renderizados eficientemente, garantindo uma performance estável mesmo com grandes volumes de dados.
- Aplicações de comércio eletrônico: exibir produtos em categorias ou listas de produtos recomendados é outra área onde o FlatList se destaca, proporcionando uma experiência de usuário suave e rápida.

Como implementar - Configuração inicial do projeto

1. **Criar projeto React Native:** configure um novo projeto React Native utilizando o CLI do React Native. Certifique-se de que todas as dependências estão instaladas e o ambiente de desenvolvimento está configurado.

Definindo os dados

2. **Dados da lista:** defina uma estrutura de dados que será renderizada pela FlatList. Isso pode ser uma array de objetos, onde cada objeto representa um item da lista.

Configurando a FlatList

3. **Componente FlatList:** importe e configure a FlatList no seu componente principal. Utilize a propriedade data para passar a lista de dados e a propriedade renderItem para definir como cada item será renderizado.

Personalizando Itens da Lista

4. **Renderização de itens:** defina a função renderItem para personalizar a aparência dos itens da lista. Utilize keyExtractor para garantir que cada item tenha uma chave única, necessária para a eficiência do React.

Funcionalidades adicionais

5. **Pull-to-Refresh:** implemente a funcionalidade de pull-to-refresh para permitir que o usuário atualize a lista de dados arrastando-a para baixo.
6. **Infinite scrolling:** configure o carregamento de mais dados conforme o usuário rola a lista, utilizando a propriedade onEndReached e onEndReachedThreshold.

Exemplo de implementação

Estrutura do projeto:

```
my-react-native-app/  
|-- src/  
| |-- components/  
| | |-- Item.js  
| |-- screens/  
| | |-- ListScreen.js  
| |-- App.js  
|-- package.json
```

Component item:

```
import React from 'react';  
import { View, Text, StyleSheet } from 'react-native';  
  
const Item = ({ title }) => {
```

```
return (  
  <View style={styles.item}>  
    <Text style={styles.title}>{title}</Text>  
  </View>  
);  
};  
  
const styles = StyleSheet.create({  
  item: {  
    padding: 20,  
    borderBottomWidth: 1,  
    borderBottomColor: '#ccc'  
  },  
  title: {  
    fontSize: 18  
  }  
});  
  
export default Item;
```

ListScreen:

```
import React, { useState, useEffect } from 'react';  
import { FlatList, StyleSheet, SafeAreaView } from 'react-native';  
import Item from '../components/Item';  
  
const ListScreen = () => {  
  const [data, setData] = useState([]);  
  const [refreshing, setRefreshing] = useState(false);  
  
  useEffect(() => {  
    fetchData();  
  }, []);  
  
  const fetchData = () => {  
    // Simula a obtenção de dados de uma API  
    setData([  
      { id: '1', title: 'Item 1' },  
      { id: '2', title: 'Item 2' },  
      { id: '3', title: 'Item 3' },  
    ]);  
  };  
};
```



```
const handleRefresh = () => {
  setRefreshing(true);
  fetchData();
  setRefreshing(false);
};

const loadMoreData = () => {
  // Simula a obtenção de mais dados
  setData([...data, { id: (data.length + 1).toString(), title: `Item
${data.length + 1}` }]);
};

return (
  <SafeAreaView style={styles.container}>
    <FlatList
      data={data}
      renderItem={({ item }) => <Item title={item.title} />}
      keyExtractor={item => item.id}
      onRefresh={handleRefresh}
      refreshing={refreshing}
      onEndReached={loadMoreData}
      onEndReachedThreshold={0.5}
    />
  </SafeAreaView>
);
};

const styles = StyleSheet.create({
  container: {
    flex: 1,
    marginTop: 20,
  },
});

export default ListScreen;
```

App.js:

```
import React from 'react';
import { SafeAreaView, StyleSheet } from 'react-native';
import ListScreen from '../screens/ListScreen';
```

```
const App = () => {  
  return (  
    <SafeAreaView style={styles.container}>  
      <ListScreen />  
    </SafeAreaView>  
  );  
};  
  
const styles = StyleSheet.create({  
  container: {  
    flex: 1,  
  },  
});  
  
export default App;
```

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, aprendemos sobre a importância e a implementação da FlatList em aplicações React Native. Discutimos como a FlatList melhora a performance ao renderizar grandes listas de dados de forma eficiente e implementamos uma FlatList para renderizar uma lista de itens, personalizando a aparência dos itens e utilizando funcionalidades adicionais como pull-to-refresh e infinite scrolling. Além disso, abordamos boas práticas para garantir a eficiência, acessibilidade e responsividade da FlatList.

Concluimos que a FlatList é uma ferramenta essencial para qualquer aplicação React Native que precise renderizar listas de dados de forma performática e organizada.

REFERÊNCIAS

CUNHA, A. React Native: o que é e como criar apps nativos com essa tecnologia? **Alura**. 2023. Disponível em: <https://www.alura.com.br/artigos/react-native>. Acesso em: 14 ago. 2024.

LINDER, A. **Mastering React Native: Full Stack Mobile App Development with React Native**. [s.l.]: Packt Publishing, 2020.

REACT NATIVE. Documentação oficial. **React Native**. [s.d.]. Disponível em: <https://reactnative.dev/docs/flatlist>. Acesso em: 19 jul. 2024.

REACT NATIVE. Handling Touches. **React Native**. 2023. Disponível em: <https://reactnative.dev/docs/handling-touches>. Acesso em: 14 ago. 2024.

PALAVRAS-CHAVE

React Native. Front-end. Layout. Headers.

EMENDAS

POS TECH